

APRIL 2026

THESIS PROPOSAL

Performance Evaluation of CP-ABE Based Access Control for MQTT
Against Lightweight Alternatives in Constrained
IoT Environments

PROFESSOR

LETTERIO GALLETTA

STUDENT

KEVIN SHEMILI

CONTENTS

1. Introduction	3
1.1. Research Questions	3
2. Baseline Cryptographic Schema.....	4
3. Alternative Cryptographic Schemes	4
4. Evaluation Metrics	6
5. Experimental Testbed	7
6. References	8

1.Introduction

The Message Queuing Telemetry Transport (MQTT) protocol is a widely deployed publish / subscribe standard for Internet of Things & cyber-physical systems, valued for its lightweight design & suitability for constrained devices.

In typical deployments, confidentiality & access control are enforced through transport-layer protection (TLS) & broker-side authorization mechanisms such as topic-based access control lists. This model assumes the broker is a trusted component able to observe message payloads in plaintext, and provides only coarse-grained access control. Furthermore, any client authorized to subscribe to a topic can read every message published on it. Both of these considerations might not always be desirable.

A client-side payload-level security layer based on Ciphertext-Policy Attribute-Based Encryption (CP-ABE) addresses both these limitations by encrypting payloads end-to-end and embedding fine-grained access policies directly into the ciphertext itself. However, it's not only good news, as CP-ABE introduces significant cryptographic overhead, raising practical questions about its viability on resource-limited devices.

This thesis therefore evaluates the performance & practicality of different payload-level security mechanisms for MQTT in constrained IoT-like environments. A CP-ABE + AES-GCM wrapper is implemented as the proposed solution and compared against relevant alternatives.

Furthermore, all schemes are evaluated under progressively tighter Docker-imposed CPU and memory constraints that simulate the operating conditions of constrained IoT hardware.

Hence, the goal is not to propose new cryptographic primitives, but to characterize the trade-offs each scheme makes between security expressiveness & operational cost, and to determine under which conditions fine-grained attribute-based access control remains a practical choice.

1.1. Research Questions

We aim at answering the following research questions:

- 1) Is CP-ABE with AES-GCM practical for IoT devices compared to other alternatives?
- 2) How does the overhead of CP-ABE scale as access policy complexity increases?
- 3) At what message rate does CP-ABE's cryptographic overhead make it unsustainable for constrained IoT devices, and how does this ceiling compare to other alternatives?

2. Baseline Cryptographic Schema

The baseline system will be a client-side security layer for MQTT based on CP-ABE + AES-GCM, whose purpose is to protect MQTT payloads end-to-end while keeping the broker unchanged. The broker remains a standard MQTT broker, such as Mosquitto, and only forwards encrypted data.

The system will be implemented in Go as a wrapper around a standard MQTT client library & applications will interact with this wrapper instead of directly publishing plaintext messages. When a publisher sends a message, the wrapper encrypts the payload before publication. When a subscriber receives a message, the wrapper attempts to decrypt it before delivering it to the application.

Solution will feature a hybrid cryptographic design, CP-ABE is used to enforce fine-grained access control through policies over attributes, while AES-GCM is used to encrypt the actual MQTT payload efficiently. CP-ABE protects a symmetric session key, while AES-GCM protects payload itself, thus avoiding direct CP-ABE use on large messages, which would be inefficient. The CP-ABE component is implemented using the TKN20 scheme^[1], provided by Cloudflare's CIRCL library^[2]. TKN20 supports conjunctive access policies over key-value attributes, and its overhead measurably scales with the number of attributes in a policy, allowing us to directly answer [RQ2](#).

This baseline system is the wrapper against which all alternative schemes will be evaluated. It will be a consolidation of an already existing solution which was developed during the bootcamp held in Lucca this year.

3. Alternative Cryptographic Schemes

We will evaluate the baseline against a set of alternative MQTT security schemes. These alternatives are selected to cover different points in the design space: transport-layer security, symmetric payload encryption, compact serialization and group-based access control.

1) TLS Only

This scheme represents the standard MQTT deployment model. Communication between clients and broker is protected using TLS, while authorization is handled by the broker through credentials and topic-based ACLs. Payloads are not encrypted end-to-end,

meaning the broker can still observe message contents. To be implemented by using Paho client with TLS enabled.

2) PSK + AES-GCM

The scheme uses a single pre-shared symmetric key to encrypt MQTT payloads with AES-GCM. The encrypted payload, nonce, and metadata are serialized using a JSON envelope with Base64-encoded binary fields. Relevant libraries are the “[CRYPTO/AES](#)” & “[CRYPTO/CIPHER](#)”.

3) Group Key per Topic

In this scheme, each MQTT topic is associated with a shared AES-GCM key. Subscribers authorized for a topic receive the corresponding group key and can decrypt messages published on that topic. Access control is therefore based on topic membership rather than attributes. Same libraries as PSK, “[CRYPTO/AES](#)” & “[CRYPTO/CIPHER](#)”, with only structural difference being that keys are stored and looked up per topic rather than globally.

4) Hybrid RSA + AES-GCM

This scheme uses hybrid public-key encryption. The payload is encrypted once with AES-GCM using a fresh session key, and the session key is then encrypted separately for each authorized subscriber using RSA. The envelope contains one encrypted payload and multiple RSA-encrypted copies of the session key. Again, handled by “[CRYPTO/AES](#)”, “[CRYPTO/CIPHER](#)” & “[CRYPTO/RSA](#)” libraries. A relevant comparison, as it is similar in spirit to CP-ABE's session key protection, but without any policy expressiveness.

5) CP-ABE + AES-GCM with Binary Envelope (CBOR)

Change baseline to serialize envelope using CBOR rather than JSON. This allows for the isolation of the cost of envelope serialization and allows a direct comparison against the baseline to determine how much overhead the JSON format contributes. To that end we will make use of the “[GITHUB.COM/FXAMACKER/CBOR/V2](#)” library^[3].

4. Evaluation Metrics

The collection & processing of quantitative metrics is fundamental to be able to answer our RQs. Metrics will be gathered programmatically, so application-level instrumentation in order to expose relevant KPIs.

METRIC	PURPOSE
ABE Encryption Time	Separate ABE specific cost from total encryption cost
Total Encryption Time	As name implies
ABE Decryption time	Isolates ABE decryption cost, considering mathematics involved may differ substantially from encryption cost
Total Decryption time	As name implies
E2E Latency	Total observable delay from publish to plaintext delivery, thus capturing full pipeline cost, not just cryptography
Encrypted Payload Size	Bandwidth overhead per message
Actual Achieved Publish Rate	Identifies the publisher-side throughput ceiling
Actual Achieved Decrypt Rate	Identifies the subscriber-side throughput ceiling
Publisher CPU & Memory Utilization	Observe CPU & Memory saturation, helping contextualize per-message costs
Subscriber CPU & Memory Utilization	Same rationale on subscriber side

LIMITATION: Another very common KPI seen in many papers, is energy consumption which is relevant to battery powered constrained devices. However, given our [testbed](#), this cannot be meaningfully measured.

5. Experimental Testbed

All experiments will be conducted on a single host machine running Docker, with each logical component (authority, broker, publisher & subscriber), deployed as a separate container.

Given our lack of actual physical IoT hardware, Docker's resource constraint mechanisms serve as a best-effort approximation of constrained device conditions. Specifically, the container's CPU & memory limits will be derived from actual hardware specifications of representative real-world IoT devices (Raspberry Pi Zero 2W, ESP32 class of microcontrollers etc.), in order to ensure that the imposed constraints reflect conditions a practitioner would plausibly encounter, rather than arbitrary values.

On the other hand, the authority & broker containers will be left unconstrained, as they represent infrastructure components rather than IoT endpoints. Only publisher & subscriber containers are resource-constrained, as these represent the IoT devices whose practicality is under evaluation.

LIMITATION: Of course, although the containerization procedure is going to help us in obtaining more accurate results, things are still running on top of the host machine. Thus, the approach cannot fully replicate the behavior of bare metal embedded hardware, which have a plethora of architectural differences in instruction set architecture, cache hierarchy etc.

TO BE DECIDED: Several aspects still remain open, here I try to group main ones to make the scope unambiguous:

- The actual mapping of the IoT hardware specifications into Docker resource constraints. Translating the capabilities of embedded devices into Docker `--cpus` and `--memory` limits is at best an approximation, given the architectural gap.
- Given that container communications on a single host does not traverse a real physical link, therefore do not pass through the TCP/IP stack, we may very well get a best scenario latency, which is not realistic for IoT environments. Under consideration the utilization of “**TC NETEM**”^[4], to introduce artificial latency, jitter, or packet loss between containers. This will have a direct effect on the measured E2E latency KPI.

6. References

[1] Tomida, J., Kawahara, Y. & Nishimaki, R. Fast, compact, and expressive attribute-based encryption. *Des. Codes Cryptogr.* 89, 2577–2626 (2021). <https://doi.org/10.1007/s10623-021-00939-8>

[2] <https://github.com/cloudflare/circl>

[3] <https://pkg.go.dev/github.com/fxamacker/cbor/v2>

[4] <https://man7.org/linux/man-pages/man8/tc-netem.8.html>